EWB COURSES | INTERNSHIP PROGRAMME

INTERNSHIP PROJECT REPORT

AWS VPC Peering

Private connectivity between two Amazon VPCs on AWS Cloud

Project Description

This project establishes private, bidirectional communication between two isolated Amazon Virtual Private Clouds - **Test-VPC (10.0.0.0/16)** and **Prod-VPC (192.168.0.0/16)** - in the AWS US East (Ohio) region. It combines a VPC peering connection, route tables, security groups, Internet Gateways and a NAT Gateway with an Elastic IP, and is verified with ping tests between EC2 instances over their private IP addresses.

Student Name	Prince Kumar
Institute	National Institute of Advanced Manufacturing Technology (NIAMT), Ranchi
Course	B.Tech in Computer Science and Engineering
Project Title	AWS VPC Peering
Project Domain	AWS Cloud Networking
Organization	EWB Courses
Email	princekumar2005itaha@gmail.com

VPC | VPC Peering | EC2 | Route Tables | Security Groups | NAT Gateway

1. Abstract, Introduction and Objectives

Abstract

This project demonstrates how two logically isolated networks on Amazon Web Services can communicate privately using **VPC Peering**. Two VPCs were created in the US East (Ohio) region (us-east-2): **Test-VPC** with CIDR 10.0.0.0/16 and **Prod-VPC** with CIDR 192.168.0.0/16. An EC2 instance was launched in each VPC, a peering connection named **Prod-Test-Connection** was created and reached the *Active* state, and routes were added to both route tables so that each VPC can reach the other's address range. Ping tests between the private IP addresses 10.0.0.139 and 192.168.0.131 succeeded in both directions with round-trip times of roughly 1.3 ms. The Test-VPC additionally contains a NAT Gateway with an Elastic IP, and both VPCs have an Internet Gateway.

Introduction

Amazon Web Services (AWS) is a cloud platform that provides on-demand compute, storage and networking. Its networking foundation is **Amazon Virtual Private Cloud (VPC)**: a logically isolated virtual network in which the user chooses the IP range, creates subnets, attaches gateways and controls traffic with route tables and security groups. By default, resources in one VPC cannot reach resources in another.

VPC Peering is a networking connection between two VPCs that lets them exchange traffic using private IPv4 addresses, as if they were on the same network. Traffic stays on the AWS network and does not traverse the public internet, and no extra gateway, VPN or separate appliance is required. Real environments commonly separate workloads - for example Production and Test - into different VPCs for isolation and security, yet still need controlled connectivity between them. This project was undertaken to build and verify exactly that pattern hands-on, and to understand the supporting routing, security and cost considerations.

Project Objectives

- Create two VPCs with non-overlapping CIDR blocks (10.0.0.0/16 and 192.168.0.0/16) and configure their subnets.
- Launch EC2 instances in each VPC and make them reachable for administration and testing.
- Create and accept a VPC peering connection between **Test-VPC** and **Prod-VPC**.
- Configure route tables on both sides so traffic for the peer VPC is sent through the peering connection.
- Configure security groups and gateways (Internet Gateway, NAT Gateway with Elastic IP).
- Verify private connectivity in both directions using ping tests, and understand the cost impact of NAT Gateways and Elastic IPs.

Expected Practical Outcome

Two instances in separate VPCs that can exchange traffic using only their private IP addresses, supported by a documented network design, correct routing on both sides, and evidence of successful connectivity.

2. Technical Background and AWS Services

The table below summarises the AWS concepts used in this project and the specific role each plays in the implementation.

Concept	What it is	Role in this project
Amazon VPC	Logically isolated virtual network with its own IP range, subnets, gateways and routing.	Two VPCs: Test-VPC (10.0.0.0/16) and Prod-VPC (192.168.0.0/16) in us-east-2.
CIDR blocks / IP addressing	Classless notation (e.g. 10.0.0.0/16) defining the private address range of a VPC or subnet. /16 gives 65,536 addresses; /24 gives 256 (5 reserved by AWS).	Ranges do not overlap, which is mandatory for peering. Subnets are /24 blocks carved from each VPC range.
Public / private subnets	A public subnet has a route to an Internet Gateway; a private subnet has no direct route to the internet.	Test-Public-Subnet (10.0.0.0/24), Test-Private-Subnet (10.0.1.0/24) and Prod-Public-Subnet (192.168.0.0/24).
EC2 instances	Virtual servers in a chosen subnet and Availability Zone.	PROD-INSTANCE and TEST-INSTANCE (t2.micro, Ubuntu) act as the endpoints for connectivity tests.
VPC Peering	A one-to-one, non-transitive connection between two VPCs, requested by one side and accepted by the other.	**Prod-Test-Connection** (pcx-02a24cc88576481c1) links the two VPCs within one account and region.
Route tables	Rules (destination CIDR -> target) deciding where subnet traffic goes.	Test-RT sends 192.168.0.0/16 to the peering connection; Prod-rt sends 10.0.0.0/16 to it. Both sides need a route.
Security groups	Stateful, instance-level virtual firewalls with allow rules.	Test-SG and Prod-SG control inbound access (SSH/ICMP) to the instances.
Internet Gateway	Horizontally scaled gateway allowing a VPC to reach and be reached from the internet.	Test-IGW and Prod-igw provide public access to the public subnets (used for administration).
NAT Gateway + Elastic IP	Lets private resources initiate outbound internet connections; it needs a static public IPv4 (Elastic IP).	Test-NAT (10.0.0.173) with Elastic IP 18.223.227.196 provides outbound access for the Test-VPC.
Bastion Host	A hardened public instance used as a jump server to private instances.	**Not used** in this project; the repository shows no bastion host.

How the components work together

An EC2 instance sends a packet to the other VPC's private IP. The **route table** of its subnet matches the destination CIDR and forwards the packet to the **peering connection**. AWS delivers it into the peer VPC, where the local route and the destination **security group** decide whether it is accepted. The reply uses the peer's route table the same way. Because a peering connection is only a path - not a router - **both** VPCs need a route to each other and both security groups must allow the traffic. Peering is also **non-transitive**: the connection does not give either VPC access to the other's Internet Gateway or NAT Gateway, so each VPC keeps its own gateway for internet access.

Key design rule: peered VPCs must not have overlapping CIDR blocks. Here 10.0.0.0/16 (Test) and 192.168.0.0/16 (Prod) are fully disjoint, so the peering connection can be created and unambiguous routes can be installed.

3. Project Architecture and Network Design

The design places the two VPCs side by side in **us-east-2 (Ohio)**, joined by a single VPC peering connection. Test-VPC contains a public subnet, a private subnet and a NAT Gateway; Prod-VPC contains one public subnet. Each VPC has its own Internet Gateway for administrative access, while all VPC-to-VPC traffic travels privately through **pcx-02a24cc88576481c1**.

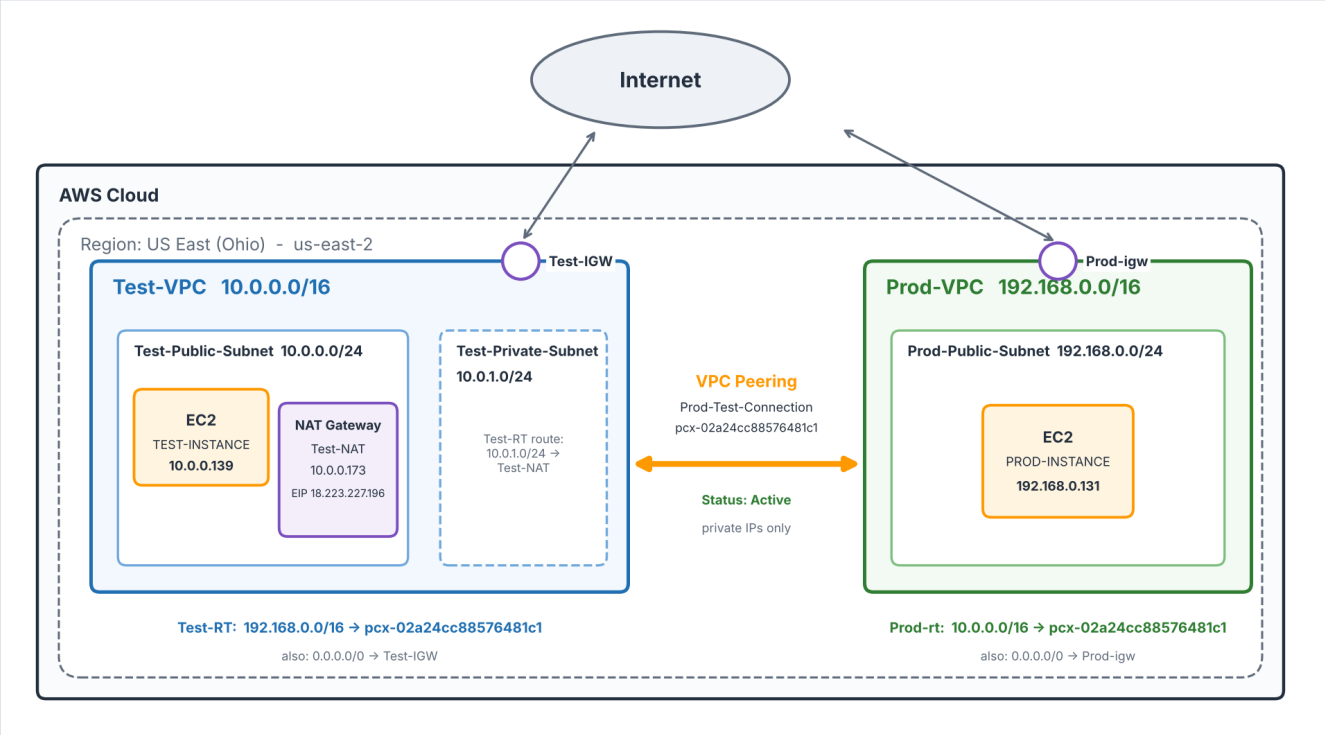


Figure 1. *Network architecture of the implemented project, drawn from the console evidence in the repository (resource names, CIDR blocks and IPs as shown in the screenshots).*

Component	Name	CIDR / IP address	AWS identifier / detail
VPC	Test-VPC	10.0.0.0/16	vpc-0227fbfb580210983
VPC	Prod-VPC	192.168.0.0/16 (local route in Prod-rt)	vpc-08d0a5a37d93c0a5d
Subnet	Test-Public-Subnet	10.0.0.0/24	subnet-097383d8f4f2e3cba
Subnet	Test-Private-Subnet	10.0.1.0/24	subnet-0ea1a718440ee3b4f
Subnet	Prod-Public-Subnet	192.168.0.0/24	subnet-0439dc5459b10aade
EC2	TEST-INSTANCE	Private 10.0.0.139 / Public 3.148.209.171	i-02f27c656d1f22cd7, t2.micro, us-east-2a
EC2	PROD-INSTANCE	Private 192.168.0.131 / Public 18.223.133.83	i-08c21b21f8e2b3ea4, t2.micro, us-east-2c
VPC Peering	Prod-Test-Connection	Requester Test-VPC, Accepter Prod-VPC	pcx-02a24cc88576481c1, Active
Internet Gateway	Test-IGW / Prod-igw	Attached to Test-VPC / Prod-VPC	igw-05a65e6eb42760322 / igw-05cb13269af761163
NAT Gateway	Test-NAT (public)	Private 10.0.0.173 / EIP 18.223.227.196	nat-090ffb7e23421f48f, Available
Security Group	Test-SG / Prod-SG	Test-VPC / Prod-VPC	sg-0b17700f1e32b5774 / sg-0ddcdb4c10a82d0ec

Table 1. IP addressing and network configuration (values taken from the repository screenshots).

4. Implementation and Configuration

Screenshots in this report are reproduced unmodified from the project repository. They display an AWS console account identifier that differs from the author's name; this will be reconciled with the author's own implementation records before final submission.

Step 1 - VPCs and subnets

Configured: Test-VPC (10.0.0.0/16) with Test-Public-Subnet (10.0.0.0/24) and Test-Private-Subnet (10.0.1.0/24); Prod-VPC (192.168.0.0/16) with Prod-Public-Subnet (192.168.0.0/24). **Why:** Each environment needs its own isolated address space, and the two ranges must not overlap for peering to be possible. **Contribution:** Defines the network foundation on which every other resource depends.

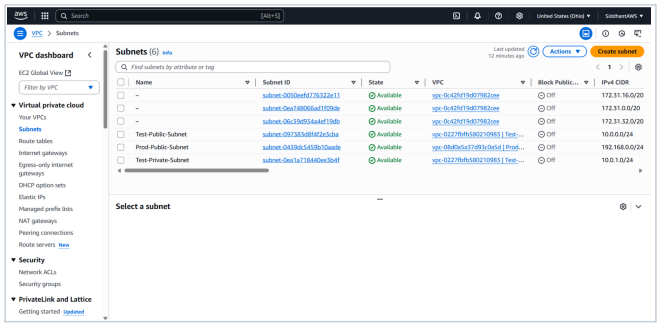


Figure 2. Subnets: Test-Public (10.0.0.0/24), Test-Private (10.0.1.0/24) and Prod-Public (192.168.0.0/24), all Available.

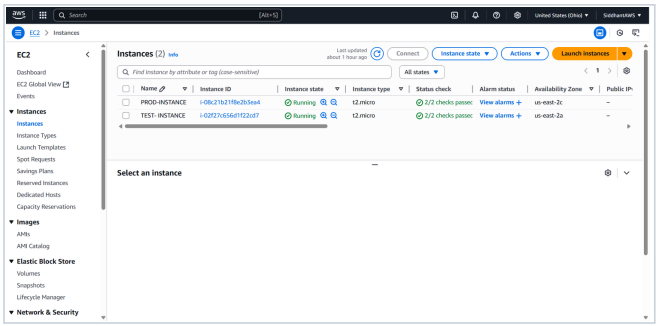


Figure 3. EC2: PROD-INSTANCE (us-east-2c) and TEST-INSTANCE (us-east-2a), t2.micro, both Running with 2/2 status checks passed.

Step 2 - Internet Gateways

Configured: Test-IGW and Prod-igw were created and attached to Test-VPC and Prod-VPC; default routes (0.0.0.0/0) to each IGW were added in Test-RT and Prod-rt. **Why:** The public subnets need an internet path so the instances can be reached for administration. **Contribution:** Makes the public subnets genuinely public and allows SSH/console access to the instances.

Step 3 - EC2 instances

Configured: TEST-INSTANCE (10.0.0.139) in Test-Public-Subnet and PROD-INSTANCE (192.168.0.131) in Prod-Public-Subnet, both t2.micro running Ubuntu, with security groups Test-SG and Prod-SG. **Why:** Endpoints are needed to generate and receive traffic across the peering link. **Contribution:** They are the source and destination of the connectivity tests.

Step 4 - VPC peering request and acceptance

Configured: Peering connection **Prod-Test-Connection** (pcx-02a24cc88576481c1): requester Test-VPC (10.0.0.0/16), accepter Prod-VPC; both VPCs are in the same account and region and the connection shows *Active*. **Why:** A peering connection must be requested by one VPC and accepted by the other before any traffic can flow. **Contribution:** Provides the private network path between the two VPCs.

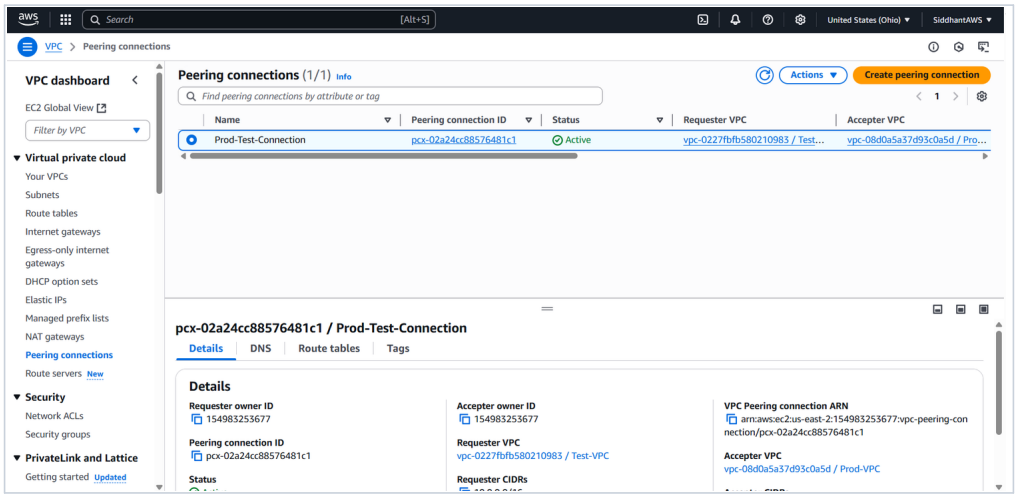


Figure 4. Peering connection Prod-Test-Connection in the Active state (requester Test-VPC, accepter Prod-VPC).

Step 5 - Route tables

Configured: Test-RT (associated with Test-Public-Subnet): 0.0.0.0/0 to Test-IGW, 10.0.0.0/16 local, **192.168.0.0/16 to the peering connection**, and 10.0.1.0/24 to the NAT Gateway. Prod-rt (associated with Prod-Public-Subnet): 0.0.0.0/0 to

Prod-igw, 192.168.0.0/16 local, **10.0.0.0/16 to the peering connection**. **Why:** Peering alone does not forward traffic; each side needs an explicit route for the other VPC's CIDR. **Contribution:** These two routes are what actually make the cross-VPC communication work in both directions.

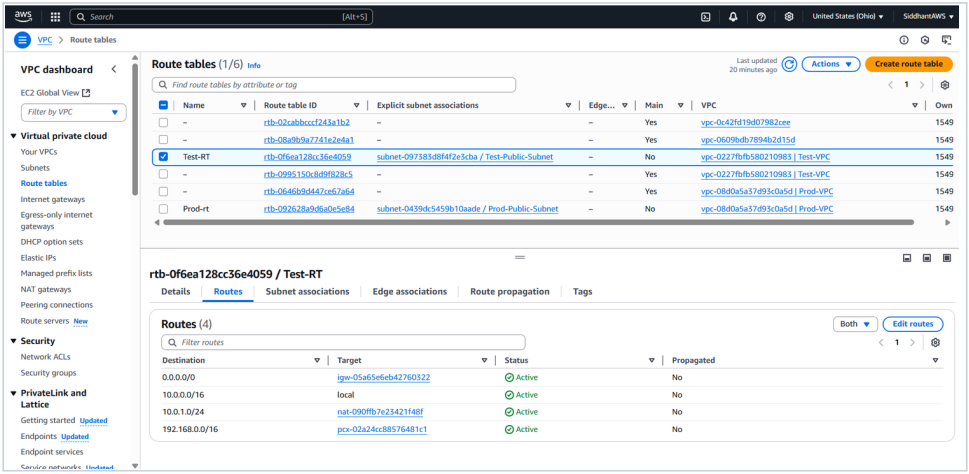


Figure 5. Test-RT routes: 192.168.0.0/16 targets pcx-02a24cc88576481c1; 0.0.0.0/0 targets the Internet Gateway.

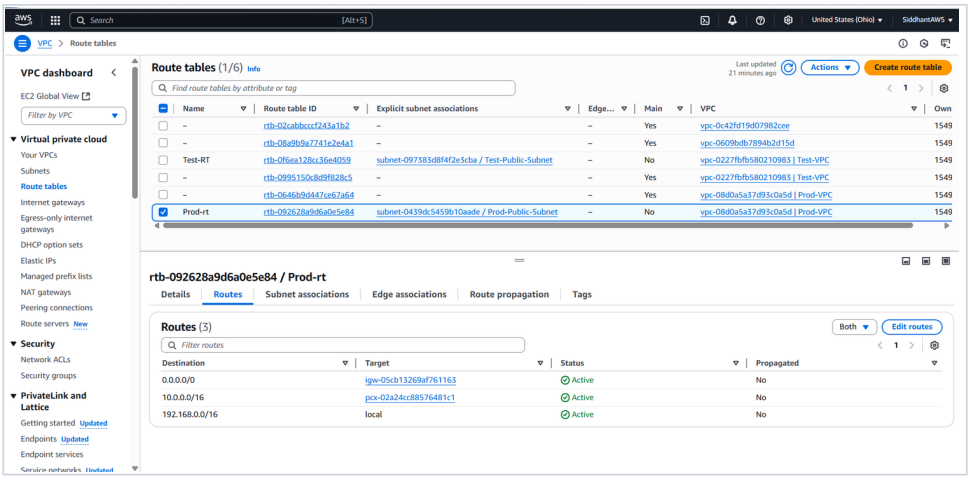


Figure 6. Prod-rt routes: 10.0.0.0/16 targets pcx-02a24cc88576481c1; 0.0.0.0/0 targets the Internet Gateway.

Step 6 - Security groups

Configured: Test-SG (sg-0b17700f1e32b5774, Test-VPC) and Prod-SG (sg-0ddcdb4c10a82d0ec, Prod-VPC), both created with the EC2 launch wizard. Test-SG shows 2 inbound and 1 outbound rule entries. **Why:** Security groups decide which traffic the instances accept; ICMP (ping) and SSH must be allowed for testing and access. **Contribution:** Completes the access path started by routing; the successful ping tests indicate ICMP was permitted on both instances.

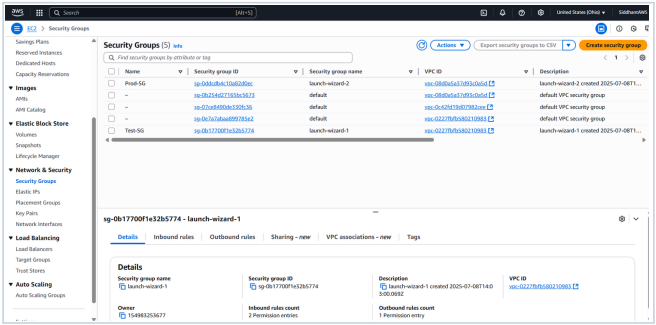


Figure 7. Security groups Test-SG and Prod-SG (individual rule contents are not captured in the repository screenshot).

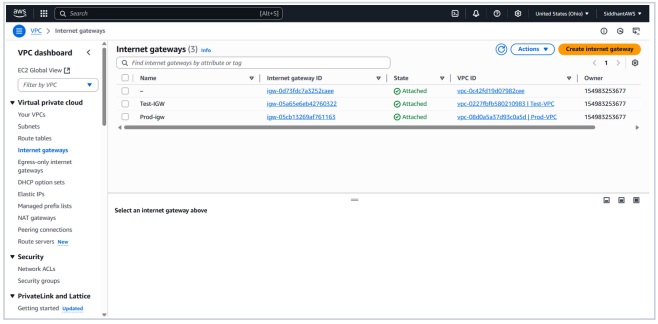


Figure 8. Internet Gateways Test-IGW and Prod-igw, both Attached to their respective VPCs.

Step 7 - NAT Gateway and Elastic IP

Configured: Test-NAT (nat-090ffb7e23421f48f, connectivity type Public, state Available, private IP 10.0.0.173) with Elastic IP 18.223.227.196 (eipalloc-02420a7d767e192f2). **Why:** The NAT Gateway enables outbound-only internet access for Test-VPC's private resources, and it requires a static public address. **Contribution:** Adds secure outbound connectivity; it also introduces ongoing cost (see Section 7).

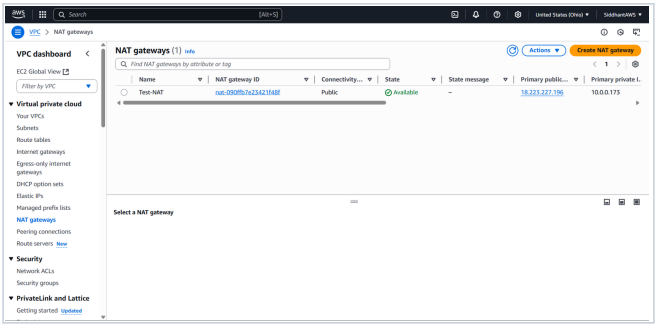


Figure 9. *NAT Gateway Test-NAT: Available, public connectivity, EIP 18.223.227.196.*

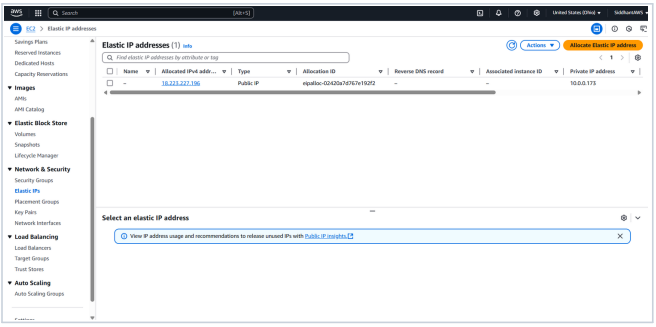


Figure 10. *Elastic IP 18.223.227.196 allocated for the NAT Gateway (private IP 10.0.0.173).*

5. Testing, Verification and Results

Verification relies on the console evidence and terminal captures stored in the repository. Each item below states only what the evidence demonstrates.

Check	Evidence in repository	Result shown
Peering connection status	Peering Connections screenshot (Fig. 5)	Prod-Test-Connection is **Active**; requester Test-VPC, accepter Prod-VPC.
Route table verification	Test-RT and Prod-rt screenshots (Figs. 6-7)	Each route table contains a route for the peer CIDR targeting pcx-02a24cc88576481c1, status Active.
EC2 status	Instances screenshot (Fig. 3)	Both instances Running, 2/2 status checks passed.
Private-IP ping, Test to Prod	Terminal capture (Fig. 8)	From TEST-INSTANCE (10.0.0.139) to 192.168.0.131: 12 replies, 64 bytes, ttl=64, time 1.25-1.38 ms.
Private-IP ping, Prod to Test	Terminal capture (Fig. 9)	From PROD-INSTANCE (192.168.0.131) to 10.0.0.139: 8 replies, 64 bytes, ttl=64, time 1.29-1.59 ms.
SSH access	No dedicated SSH log; commands were run in a browser-based terminal session in the AWS console.	Terminal access to both instances is evidenced by the ping captures.

Communication flow

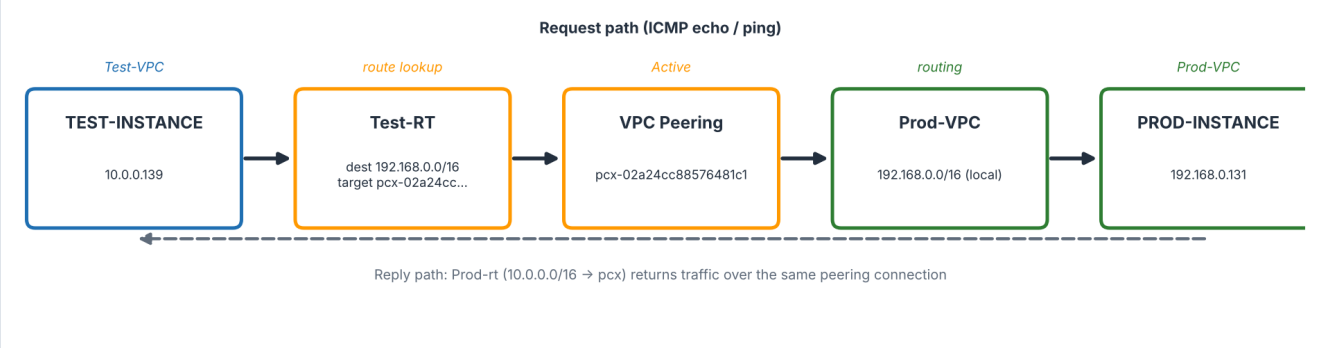


Figure 11. *Packet path for a ping from TEST-INSTANCE to PROD-INSTANCE and the return path through Prod-rt.*

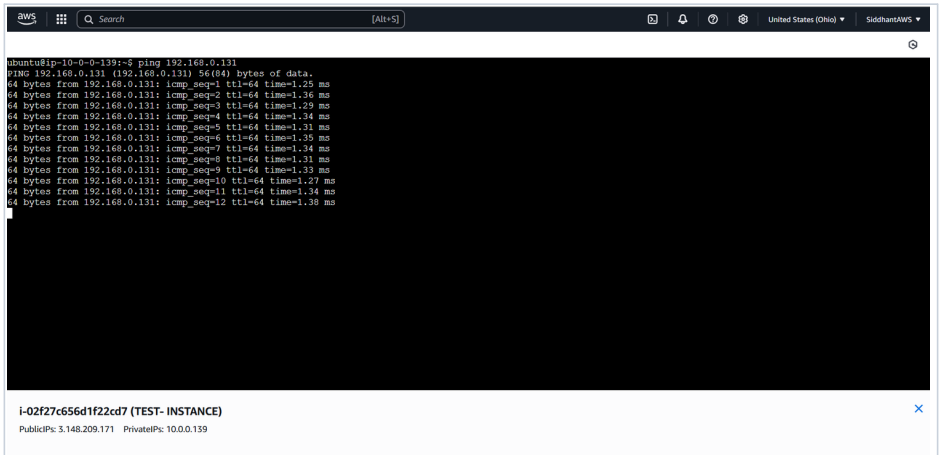


Figure 12. *Ping from TEST-INSTANCE (10.0.0.139) to PROD-INSTANCE private IP 192.168.0.131 - replies received over the peering connection.*

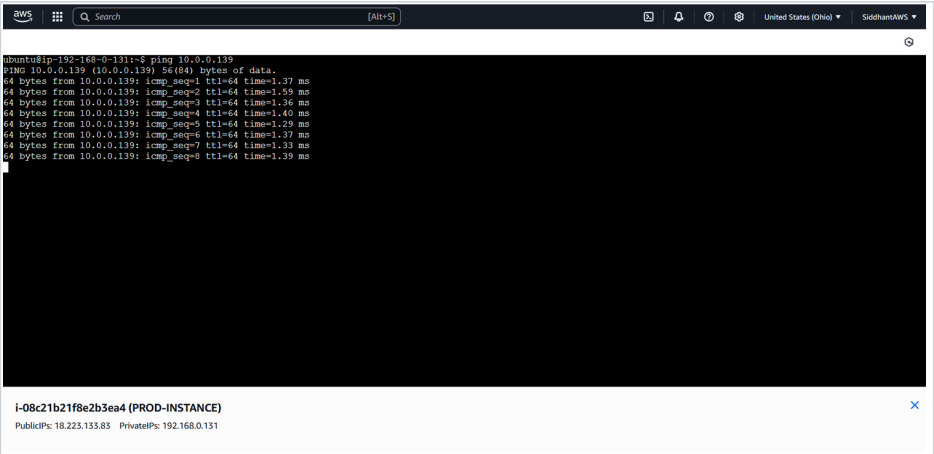


Figure 13. *Ping from PROD-INSTANCE (192.168.0.131) to TEST-INSTANCE private IP 10.0.0.139 - replies received in the reverse direction.*

Results

Both captures show uninterrupted replies with no timeouts and a consistent latency of about 1.3-1.6 ms, which confirms that routes, the peering connection and the security groups are all correct and that traffic flows in **both directions** using private addresses. The captures were stopped manually, so no end-of-run packet-loss summary is shown, and only ICMP was tested; no TCP application traffic is evidenced.

6. Security, Challenges and Troubleshooting

Security considerations

Area	Observation in this project	Good practice
Security groups	Test-SG and Prod-SG are attached to the instances; Test-SG has 2 inbound and 1 outbound entries. Individual rules are not visible in the evidence.	Allow only required ports; for cross-VPC traffic reference the peer CIDR (10.0.0.0/16 or 192.168.0.0/16) instead of 0.0.0.0/0.
Private communication	Ping between private IPs succeeded, so VPC-to-VPC traffic uses the peering connection and does not cross the internet.	Keep workload traffic on private addresses and expose only what must be public.
SSH access	Instances have public IPs through Internet Gateways and were accessed from the console terminal.	Restrict port 22 to a known IP, use key pairs, or use Session Manager / a bastion host.
Routing	Peer routes exist on both sides; peering is non-transitive.	Use the narrowest destination CIDR needed and review route tables after changes.
Cost control	README notes NAT Gateway and Elastic IP incur charges.	Delete unused NAT Gateways and release Elastic IPs after testing.

Challenges actually documented in the project

The repository does not contain an error log or a record of specific faults. The only challenge recorded in the README is **cost management**: the author learned that the NAT Gateway and Elastic IP are not covered by the AWS Free Tier and can create unexpected charges, so unused resources must be deleted. The README also lists Cost Explorer / billing review as part of the project, but no billing screenshots are included in the repository, so none are presented here.

Observation from the evidence: Test-RT contains a route for 10.0.1.0/24 pointing to the NAT Gateway, in addition to the usual 0.0.0.0/0 route to the Internet Gateway. The route table used by Test-Private-Subnet is not shown in the screenshots, so its exact routing could not be independently verified.

General troubleshooting guide (not specific faults of this project)

Symptom	Likely cause	What to check
Peering stays in *pending-acceptance*	Request not accepted by the accepter VPC owner	Accept the request in VPC > Peering connections.
Peering cannot be created	Overlapping CIDR blocks	Compare VPC CIDRs; they must be disjoint.
Ping times out between VPCs	Missing route on one side, or ICMP blocked	Verify routes in both route tables; allow ICMP from the peer CIDR in the security group; check NACLs.
SSH to instance times out	Port 22 not allowed, no public IP, or no IGW route	Check the security group, public IP and 0.0.0.0/0 route to the IGW.
Private subnet has no internet	No route to NAT Gateway, or NAT is not in a public subnet	Check the private route table and that the NAT Gateway is Available.
Traffic through a third VPC fails	Peering is non-transitive	Create direct peering or use a Transit Gateway.

7. Applications, Learning Outcomes and Conclusion

Practical applications of VPC Peering

- **Environment separation:** connecting Production, Test and Development VPCs while keeping them isolated by default.
- **Shared services:** allowing application VPCs to reach a central VPC with databases, monitoring or logging.
- **Multi-team and multi-account networks:** linking VPCs owned by different teams or AWS accounts without public exposure.
- **Migration and integration:** private connectivity between legacy and new workloads during cloud migrations.

These patterns are everyday work for cloud and DevOps engineers, which makes the skills practised here directly relevant to real infrastructure.

Skills and knowledge gained

- Designing non-overlapping CIDR plans and subnets across VPCs.
- Creating and accepting VPC peering connections and understanding why they are non-transitive.
- Configuring route tables, Internet Gateways, NAT Gateway and Elastic IP.
- Using security groups and verifying connectivity with ping from the Linux terminal.
- Recognising AWS cost traps (NAT Gateway, Elastic IP) and the need to clean up unused resources.

What the project demonstrates

The evidence shows an Active peering connection, correct reciprocal routes, two running instances and successful bidirectional ping over private IP addresses (10.0.0.139 and 192.168.0.131) at about 1.3 ms latency. Together these confirm that Test-VPC and Prod-VPC can communicate privately through VPC peering.

Limitations

- Only ICMP (ping) was tested; no TCP application or database traffic is evidenced.
- Security group rule details, and an instance inside Test-Private-Subnet, are not shown in the repository evidence.
- Both VPCs are in the same region and account; cross-region and cross-account peering and DNS resolution across the peering were not covered.
- The architecture diagram originally in the repository uses illustrative route-table icons; this report's diagram uses the values visible in the console screenshots.

Possible future improvements

- Define the infrastructure as code (Terraform or CloudFormation) for repeatable deployment.
- Restrict security groups to specific ports and peer CIDRs; add Network ACLs and VPC Flow Logs for monitoring.
- Replace public SSH with AWS Systems Manager Session Manager or a bastion host, and place workloads in private subnets.
- Extend to multi-VPC designs using AWS Transit Gateway and add CloudWatch alarms and cost budgets.

Conclusion

The project successfully built and verified private connectivity between two VPCs with non-overlapping address ranges using AWS VPC Peering. It showed that peering requires three aligned elements - an accepted connection, routes on both sides and permissive security groups - and highlighted the operational and cost aspects of NAT Gateways and Elastic IPs. The work provided practical, interview-relevant experience in AWS cloud networking and forms a solid base for more advanced network architectures.

References

[1] Amazon VPC User Guide - What is Amazon VPC? https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html
[2] Amazon VPC Peering Guide - What is VPC peering? https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html
[3] Amazon VPC User Guide - Route tables. https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html
[4] Amazon VPC User Guide - Security groups. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html
[5] Amazon VPC User Guide - NAT gateways. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
[6] Amazon VPC User Guide - Internet gateways. https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html